FILIÈRE MIAGE — LICENCE 2 (ANNÉE ACADÉMIQUE 2026 - 2027)

SYSTÈMES D'EXPLOITATION MULTIPROCESSEURS

Principes Fondamentaux, Architectures, Concurrence, Évolutions et Accélérateurs Spécialisés

Groupe	Groupe 1	**Document**	Rendu 1 (Synthèse & Cours)
Membres	Zida Rachidatou & Sawadogo Issiaka	**Thématique**	Système d'Exploitation Multiprocesseur

Introduction : L'impasse physique et l'avènement du multicœur

Pendant près de trois décennies, l'industrie informatique a bénéficié d'une accélération quasi automatique des traitements logiciels. Chaque nouvelle génération de processeurs augmentait la fréquence d'horloge, permettant d'exécuter le même code deux fois plus vite sans modifier une seule ligne de programme. Herb Sutter a qualifié la fin de cette période par une formule restée célèbre : la fin du « Free Lunch » [Sutter, 2005].

Cette progression constante reposait sur deux principes fondamentaux. D'une part, la loi de Moore, observée dès 1965, prévoyait le doublement de la densité de transistors sur une puce de silicium environ tous les deux ans [Moore, 1965]. D'autre part, la loi d'échelle de Dennard (1974) établissait que la baisse de tension d'alimentation associée à la réduction de taille des transistors maintenait une densité de puissance constante [Dennard et al., 1974]. La fréquence d'horloge pouvait ainsi grimper sans provoquer d'échauffement excessif.

> **ANALOGIE : L'AMÉLIORATION DE LA VOIE UNIQUE**
> Pendant des années, l'optimisation des monoprocesseurs s'est apparentée à l'amélioration d'un véhicule sur une route à une seule voie : le moteur tournait de plus en plus vite et le trajet devenait deux fois plus court sans modifier le style de conduite.

La rupture physique : les trois murs matériels

Au milieu des années 2000, la miniaturisation poussée des transistors a atteint des limites physiques fondamentales, provoquant l'effondrement de la loi de Dennard face à trois obstacles majeurs [Silberschatz et al., 2018] :

- **Le mur de la puissance (Power Wall) :** Les fuites de courant à l'échelle nanométrique entraînent un dégagement thermique massif. Dissiper cette chaleur sur une surface de quelques millimètres carrés exigerait des systèmes de refroidissement irréalistes pour le grand public.
- **Le mur de la fréquence (Frequency Wall) :** La montée en fréquence s'est arrêtée aux alentours de 3,5 à 4 GHz. Dépasser cette limite risque de détruire irrémédiablement le silicium par surchauffe.
- **Le mur de la mémoire (Memory Wall) :** La vitesse d'accès à la mémoire vive (RAM) progresse beaucoup plus lentement que la rapidité de calcul du processeur. Le processeur passe alors une part importante de son temps en attente d'instructions ou de données.

Le passage au multicœur et la définition d'un OS multiprocesseur

Face à l'impossibilité d'augmenter la vitesse d'un cœur unique, les fondeurs ont orienté leurs architectures vers la juxtaposition de plusieurs cœurs de calcul autonomes sur la même puce. Un système d'exploitation multiprocesseur se définit comme un logiciel système capable d'exploiter simultanément plusieurs unités

centrales de traitement (CPU ou cœurs), de répartir équitablement les tâches en exécution et de coordonner les accès partagés aux ressources mémoire et périphériques sans dégrader la cohérence globale [Stallings, 2018].

> **ANALOGIE : LE CONVOI LOGISTIQUE**
> Ne pouvant plus construire une voiture de course individuelle roulant à 1000 km/h, l'industrie déploie dix camions roulant chacun à 100 km/h. Le volume global de marchandises transportées augmente, mais la logistique exige un régulateur efficace pour coordonner les trajets et éviter les encombrements.

Partie 1 : Typologie des Systèmes et Modèles d'Organisation du Noyau

L'organisation interne du système d'exploitation doit adapter sa structure pour s'exécuter lui-même sur plusieurs cœurs. Plusieurs architectures d'OS coexistent selon la répartition du pouvoir de décision entre les processeurs.

1.1 Le modèle Asymétrique (ASMP - Asymmetric Multiprocessing)

Dans le modèle asymétrique, un processeur unique (appelé maître) détient l'exclusivité de l'exécution du noyau du système d'exploitation. Les autres processeurs (appelés esclaves) exécutent uniquement les applications utilisateurs. Lorsqu'un processeur esclave requiert un service système (accès disque, allocation mémoire, entrée/sortie), il adresse une requête au processeur maître [Silberschatz et al., 2018].

Le maître gère une file d'attente centralisée contenant les processus à l'état prêt et assigne directement les travaux aux esclaves. Ce modèle présente une grande simplicité de conception car il évite les conflits d'accès simultanés aux structures de données du noyau. En contrepartie, le processeur maître devient rapidement un goulot d'étranglement qui limite le passage à l'échelle. De surcroît, la panne du maître entraîne l'arrêt complet de l'ensemble de la machine.

> **ANALOGIE : LA BRIGADE AVEC UN SEUL CHEF DE CUISINE**
> Plusieurs commis préparent les ingrédients, mais un unique chef détient les clés de la réserve. Chaque commis doit attendre en file devant le chef dès qu'il lui faut un produit supplémentaire, créant un temps d'attente prolongé.

1.2 Le modèle Symétrique (SMP - Symmetric Multiprocessing)

Le modèle symétrique supprime toute hiérarchie maître-esclave : tous les processeurs sont stricts égaux et chacun a la capacité d'exécuter le code du noyau ainsi que les applications [Tanenbaum & Bos, 2015]. C'est l'architecture adoptée par l'ensemble des noyaux modernes tels que Linux, Windows et macOS.

Dans un système SMP, chaque processeur interagit avec une file d'attente globale ou possède sa propre file locale. L'exécution simultanée du noyau sur plusieurs cœurs impose d'introduire des mécanismes de verrouillage rigoureux afin d'éviter que deux processeurs ne modifient en même temps la même table du système (par exemple la liste des processus prêts ou la table des pages).

1.3 Les Clusters et Systèmes Distribués

Un cluster regroupe plusieurs ordinateurs autonomes (appelés nœuds) reliés par un réseau local à haut débit. Contrairement aux systèmes SMP où la mémoire est partagée directement par le matériel, chaque nœud d'un cluster dispose de sa propre mémoire vive, de son propre système d'exploitation et de ses propres disques durs. La coordination globale s'effectue par échange explicite de messages sur le réseau.

1.4 Protéger les structures de données du noyau en SMP

L'accès simultané des cœurs aux structures internes du noyau nécessite une protection ciblée :

- **Le verrou global du noyau (Big Kernel Lock - BKL) :** Approche historique consistant à placer un cadenas unique sur l'entrée du noyau. Un seul cœur pouvait exécuter du code noyau à un instant donné [Bovet & Cesati, 2005]. Bien que simple, le BKL transformait le SMP en système quasi asymétrique lors d'appels système fréquents. Le noyau Linux a progressivement éliminé le BKL jusqu'à sa suppression définitive dans la version 2.6.39 en 2011.
- **Les verrous à grain fin (Fine-grained locking) :** Utilisation de verrous distincts pour chaque structure de données spécifique (un verrou pour la table des fichiers ouverts, un verrou par file d'attente d'ordonnancement, etc.). Cette granularité offre un parallélisme élevé mais augmente la complexité du code noyau.

Critère de comparaison	Multiprocesseur Asymétrique (ASMP)	Multiprocesseur Symétrique (SMP)	Cluster Distribué
Exécution du noyau	Processeur maître exclusivement	Tous les processeurs indifféremment	Un noyau indépendant par nœud
Accès à la mémoire RAM	Mémoire partagée globale	Mémoire partagée globale (UMA/NUMA)	Mémoire privée par nœud (réseau)
Goulot d'étranglement	Surcharges fréquentes du maître	Contention sur les verrous du noyau	Bande passante et latence réseau
Résilience aux pannes	Faible (si le maître chute)	Élevée (reprise par les autres cœurs)	Très élevée (isolation des nœuds)
Passage à l'échelle	Limité (quelques cœurs)	Bon (dizaines à centaines de cœurs)	Massif (des milliers de nœuds)

Partie 2 : Architectures Matérielles et Gestion de la Mémoire

La topologie des connexions physiques entre processeurs et barrettes de mémoire conditionne directement les algorithmes du système d'exploitation.

2.1 Architectures à mémoire partagée : UMA vs NUMA

Dans un système à mémoire partagée, tous les processeurs partagent le même espace d'adressage physique. On distingue deux approches majeures :

- **UMA (Uniform Memory Access) :** Tous les processeurs sont reliés à la mémoire centrale par un bus système commun ou un commutateur crossbar. Le temps d'accès à n'importe quel emplacement mémoire est strictement identique pour l'ensemble des cœurs. Dès que le nombre de cœurs dépasse quelques dizaines, le bus central devient un facteur d'encombrement critique.
- **NUMA (Non-Uniform Memory Access) :** La mémoire vive est physiquement fractionnée et rattachée directement à des groupes de processeurs (appelés nœuds NUMA). Bien que l'espace d'adressage reste global et unifié, la vitesse d'accès varie : la lecture dans la mémoire locale au nœud s'effectue en quelques nanosecondes, tandis que l'accès à la mémoire distante rattachée à un autre nœud impose un passage par un bus d'interconnexion interne (comme Intel UPI ou AMD Infinity Fabric), induisant un délai supérieur.

> **ANALOGIE : LES ARMOIRES DE RANGEMENT**
> L'architecture UMA équivaut à une unique armoire centrale située au milieu de la pièce où tous les employés viennent chercher leurs dossiers. L'architecture NUMA fournit un tiroir personnel sous le bureau de chaque employé : l'accès à son propre tiroir est immédiat, alors que chercher un document sur le bureau du voisin exige de se déplacer.

2.2 Architectures à mémoire distribuée et passage de messages

Dans les architectures distribuées, aucun espace mémoire n'est partagé matériellement. Chaque processeur accède uniquement à sa mémoire privée. Toute transmission d'information requiert l'émission et la réception explicites de messages via le réseau local (interfaces MPI - Message Passing Interface). Le système d'exploitation gère la pile protocolaire et la mise en mémoire tampon des paquets.

Partie 3 : Ordonnancement des Processus et Équilibrage de Charge

L'ordonnanceur d'un système multiprocesseur doit décider quel processus attribuer à chaque cœur tout en optimisant le débit global et en réduisant la latence d'exécution.

3.1 Structure des files d'attente d'ordonnancement

Deux organisations principales existent pour maintenir les tâches prêtes à s'exécuter :

- **File d'attente globale unique :** Une liste unique rassemble l'intégralité des processus du système. Tous les cœurs viennent y prélever leur prochaine tâche. Cette approche garantit une équité parfaite mais provoque une forte contention : chaque retrait de tâche exige de verrouiller la file globale, paralysant temporairement les autres cœurs.
- **Files d'attente locales par cœur :** Chaque cœur gère sa propre liste indépendante de processus. Aucune contention de verrouillage ne survient lors des opérations courantes. En revanche, un déséquilibre peut apparaître si un cœur termine rapidement son travail pendant qu'un autre reste surchargé.

> **ANALOGIE : LES FILES D'ATTENTE DE MAGASIN**
> Une file d'attente unique (type guichet bancaire) garantit un ordre de passage strict mais crée une attente centralisée. Des files séparées devant chaque caisse de supermarché fluideifient le service, sauf si un client devant vous ralentit toute la file pendant que la caisse voisine est libre.

3.2 Stratégies d'équilibrage de charge : le Vol de Travail (Work Stealing)

Pour corriger le déséquilibre des files locales, les OS modernes implémentent l'algorithme de vol de travail (*Work Stealing*) [Tanenbaum & Bos, 2015]. Lorsqu'un cœur vider sa file locale, il prend l'initiative d'examiner la file d'un cœur voisin saturé et d'en transférer une partie des tâches prêtes vers sa propre file.

Ce rééquilibrage dynamique est tempéré par la notion d'**affinité processeur** (*CPU affinity*). Exécuter un processus sur un même cœur permet de réutiliser les données déjà présentes dans ses mémoires caches rapides (L1 et L2). Déplacer un processus vers un autre cœur entraîne une perte d'affinité cache (*cache miss*) et ralentit le démarrage initial de la tâche.

3.3 Ordonnancement adapté à la topologie NUMA

Sur les architectures NUMA, l'ordonnanceur cherche prioritairement à maintenir un processus sur le nœud processeur où sa mémoire vive a été initialement allouée (*NUMA-aware scheduling*) [Bovet & Cesati, 2005]. Si

un déplacement s'avère inévitable pour équilibrer la charge, le système d'exploitation peut enclencher une migration de pages mémoire (*page migration*) pour transférer les données vers le nœud récepteur.

Partie 4 : Synchronisation, Concurrence et Pièges Logiciels

L'exécution parallèle de plusieurs fil d'exécutions (threads) sur différents cœurs introduit des risques d'interférences appelés erreurs de concurrence.

4.1 Conditions de course et opérations atomiques matérielles

Une condition de course (*race condition*) survient lorsque deux cœurs accèdent simultanément à un même emplacement mémoire et qu'au moins un de ces accès est une écriture [Stallings, 2018]. Le résultat final dépend alors de l'ordre d'arrivée arbitraire des signaux électriques sur le bus.

Sur un monoprocesseur, il suffisait de désactiver temporairement les interruptions pour exécuter une section critique sans être interrompu. En multiprocesseur, désactiver les interruptions sur le cœur local n'empêche pas les autres cœurs d'accéder à la mémoire. La synchronisation requiert des instructions matérielles atomiques intégrées au processeur, telles que **Compare-And-Swap (CAS)** ou **Test-And-Set (TAS)**. Ces instructions verrouillent la ligne de cache mémoire pendant la durée de l'opération de lecture-modification-écriture.

```
// Exemple de boucle d'attente active basique avec CAS
while (!CompareAndSwap(&lock_variable, 0, 1)) {
    // Attente active (Spinlock)
}
// Section critique sécurisée
...
lock_variable = 0; // Libération du verrou
```

4.2 Primitives de synchronisation : Spinlocks vs Mutexes

Le choix de la méthode d'attente impacte directement l'efficacité du système :

- **Spinlock (Verrou à attente active) :** Le cœur effectue une boucle d'instruction en continu tant que le verrou n'est pas libéré. Il offre une réactivité maximale si l'attente est ultra-courte (de l'ordre de quelques nanosecondes), mais consomme inutilement des cycles CPU et de l'énergie si le verrou reste occupé.
- **Mutex (Verrou d'exclusion mutuelle) :** Si le verrou n'est pas disponible, l'OS suspend le thread et le place en état de sommeil, réattribuant immédiatement le cœur à une autre tâche. Cette méthode évite le gaspillage de ressources, mais implique un surcoût lors du changement de contexte (*context switch*).

4.3 Bugs classiques du parallélisme

- **Interblocage (Deadlock) :** Situation où deux tâches ou plus se bloquent mutuellement en attendant la libération de ressources détenues par les autres. Les quatre conditions de Coffman (exclusion mutuelle, rétention et attente, non-préemption, attente circulaire) doivent être réunies simultanément pour qu'un interblocage survienne [Coffman et al., 1971].
- **Famine (Starvation) et Inversion de priorité :** La famine désigne l'incapacité d'une tâche à accéder aux ressources en raison du passage prioritaire d'autres processus. L'inversion de priorité survient lorsqu'une

tâche de haute priorité est bloquée en attente d'un verrou tenu par une tâche de basse priorité, elle-même préemptée par une tâche de priorité intermédiaire.

- **Faux partage de cache (False Sharing) :** Ce phénomène se produit lorsque deux variables indépendantes, modifiées séparément par deux cœurs distincts, se trouvent par hasard sur la même ligne de cache mémoire (typiquement 64 octets). Les mécanismes de cohérence de cache forcent l'invalidation constante de la ligne entre les cœurs, effondrant les performances du bus sans qu'aucune erreur logique n'apparaisse dans le code [Tanenbaum & Bos, 2015].

Partie 5 : Avantages, Défis et Limites Théoriques

5.1 Avantages fondamentaux

- **Performances brutes accrues :** Capacité à exécuter plusieurs applications lourdes de manière réellement simultanée plutôt qu'en temps partagé rapide.
- **Meilleure utilisation des ressources matérielles :** Répartition dynamique des charges de travail sur l'ensemble des cœurs disponibles.
- **Haute disponibilité et résilience :** Tolérance aux pannes matérielles isolées. La défaillance d'un cœur n'entraîne pas le crash global du système, l'OS pouvant réassigner les tâches sur les cœurs sains.
- **Évolutivité (Scalabilité) :** Possibilité d'étendre la puissance de traitement par simple ajout de processeurs ou de nœuds supplémentaires.

5.2 Défis majeurs de conception

Le déploiement de systèmes multiprocesseurs fait émerger des contraintes spécifiques : la complexité d'écriture des logiciels parallèles, le surcoût d'administration (*overhead*) causé par la synchronisation et la gestion des échanges inter-processeurs, ainsi que la difficulté d'équilibrer uniformément des tâches aux besoins en ressources hétérogènes.

5.3 Limites théoriques du parallélisme : la loi d'Amdahl

La loi d'Amdahl (1967) formule la limite théorique d'accélération d'un programme sur une architecture parallèle [Amdahl, 1967]. En désignant par p la proportion du code parallélisable et par n le nombre de cœurs de calcul, l'accélération globale $S(n)$ s'exprime ainsi :

> **Gain d'accélération $S(n) = 1 / [(1 - p) + (p / n)]$**

Si un programme contient seulement 5 % de code intrinsèquement séquentiel (soit $1 - p = 0{,}05$), le gain maximal théorique est plafonné à **20**, quel que soit le nombre de cœurs ajoutés (même avec un million de cœurs). Cette loi met en évidence l'impact critique des goulots d'étranglement séquentiels.

5.4 L'évolution vers les architectures asymétriques modernes

Pour contrecarrer les limites thermiques et la consommation d'énergie, les processeurs modernes combinent sur le même composant des cœurs aux caractéristiques complémentaires (comme l'architecture ARM big.LITTLE / DynamIQ ou les cœurs P et E chez Intel) [Silberschatz et al., 2018]. L'ordonnanceur du système d'exploitation doit faire preuve d'intelligence en assignant les applications gourmandes (jeux, rendu) aux cœurs de performance (P-cores) et les tâches de fond aux cœurs d'efficacité énergétique (E-cores).

Partie 6 : Applications Pratiques et Accélérateurs Spécialisés

Les systèmes multiprocesseurs et leurs extensions spécialisées irriguent aujourd'hui tous les secteurs informatiques.

6.1 Domaines d'application principaux

- **Calcul Haute Performance (HPC) et Supercalculateurs :** Simulation numérique complexe, modélisation climatologique, physique quantique et prévisions météorologiques exigeant des clusters de milliers de nœuds.
- **Serveurs, Datacenters et Cloud Computing :** Traitement en parallèle de millions de requêtes web simultanées et gestion de bases de données distribuées.
- **Multimédia, Graphisme et Rendu :** Moteurs de jeu 3D temps réel, montage vidéo 4K/8K et calcul de scènes synthétiques.
- **Intelligence Artificielle et Apprentissage Profond :** Entraînement et inférence de réseaux de neurones massifs (LLM, vision par ordinateur).
- **Systèmes Embarqués et Robotique :** Équipements automobiles autonomes, télécommunications 5G et systèmes d'avionique.

6.2 L'intégration des puces spécialisées (GPU, NPU, TPU) et des SoC

Le rôle de l'OS s'est étendu de la simple gestion du CPU vers l'orchestration de co-processeurs hautement spécialisés [Hennessy & Patterson, 2017] :

- **GPU (Graphics Processing Unit) :** Composé de milliers de cœurs très simples optimisés pour le traitement de données en masse (architecture SIMD/SIMT). L'OS dialogue avec le GPU via un pilote spécialisé pour déléguer les calculs matriciels.
- **NPU (Neural Processing Unit) et TPU (Tensor Processing Unit) :** Circuits conçus spécifiquement pour exécuter à très basse consommation les opérations d'intelligence artificielle (multiplications de matrices). Ils équipent les smartphones modernes (Apple Neural Engine, Google Tensor) et les PC récents pour la reconnaissance vocale et le traitement d'image en local.
- **SoC (System on Chip) :** Intégration sur une seule puce de silicium du CPU hybride, du GPU, du NPU, du modem et des contrôleurs d'entrée/sortie. Sur ces puces, l'OS priorise la sobriété thermique et l'autonomie de la batterie.

Conclusion

L'évolution des systèmes d'exploitation multiprocesseurs illustre une adaptation constante du logiciel face aux contraintes physiques de la matière. De l'épuisement du modèle monoprocesseur à l'émergence des architectures multicœurs symétriques (SMP), des topologies NUMA jusqu'à la prise en charge des accélérateurs IA (GPU et NPU), l'OS s'est métamorphosé. Il ne se contente plus de répartir du temps CPU : il officie désormais comme le chef d'orchestre d'une mosaïque de ressources de calcul hétérogènes, assurant la cohésion, la sécurité et la fluidité des exécutions informatiques modernes.

Bibliographie et Références Académiques

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30, 483-485.

Bovet, D. P., & Cesati, M. (2005). *Understanding the Linux Kernel* (3rd ed.). O'Reilly Media.

Coffman, E. G., Elphick, M., & Shoshani, A. (1971). System Deadlocks. *ACM Computing Surveys*, 3(2), 67-78.

Dennard, R. H., et al. (1974). Design of ion-implanted MOSFET's with very small physical dimensions. *IEEE Journal of Solid-State Circuits*, 9(5), 256-268.

Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.

Love, R. (2010). *Linux Kernel Development* (3rd ed.). Addison-Wesley Professional.

Moore, G. E. (1965). Cramming more components onto integrated circuits. *Electronics*, 38(8), 114-117.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.

Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.

Sutter, H. (2005). The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software. *Dr. Dobb's Journal*, 30(3), 16-22.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.